

Assignment 2:Free RTOS

By Veselin Atanasov



Teacher: Karin Dieleman

Date:07-11-2024

Contents

Introduction	3
Threads.....	3
Mutex	4
Queues	6
Conclusion.....	7

Introduction

The goal of this assignment is to create a freeRTOS application on the ESP32 to control sprinklers(leds) in different schemes called scenes. This assignment is split in three parts: threads, mutex and queues. In this document I will explain the process of developing the application and explain my decisions.

Threads

For the sake of clarity a table of implemented requirements will be shown below:

Requirement	Definition
User chooses a scene	The user should be able to select a scene at any time using the serial monitor, by sending a message that corresponds to the scene number
Scenes are stored in an efficient way	The scenes are stored in a readable and executable manner(in this case arrays)
There is a function to execute scenes	A function should be in charge of interpreting the commands of the scenes and also executing them
Error handling	The program should have a way of informing the user of an unexpected error by sending a message to the user via the serial monitor
Multiple scene should be able to run simultaneously	Multiple scenes should be able to execute at the same time if the user requests multiple scenes at the same time or in short intervals

I decided to store the scenes in arrays of strings following the example structure in the assignment as working with them is readable and efficient enough for this application. To execute the scenes I created the executeScene function. This function checks every member of the array and compares the strings to the hardcoded strings. When the string from the array matches one of the hardcoded strings, the respective action is executed. For example if the string “sp1 on” is matching the first sprinkler will turn on, if the matching string is “sp1 off” the first sprinkler will turn off and so on. If the command is not recognized, the “Invalid command” message will show up on the monitor. This function just reads each

string from the scene and executes it. To test this function I just ran the executeScene function in the main loop. I ran it with each scene separately to see if the scenes are executed as expected.

After I was sure that scenes were properly executed, I started working on implementing threads. The first thread that I created, is the thread to read the serial monitor. This is a thread that needs to run constantly during runtime, so it is created in the setup and has an infinite loop to make sure it never stops running, giving the user the advantage to be able to send messages over serial on demand. This function reads the message on the serial and also creates another task depending on the message received. When the user sends a message, it will be printed on the serial monitor and the scene corresponding to the message will start. For example if the message is “1” scene 1 will start executing. This is done by a method called chooseScene. This method receives the message sent by the user as an argument and creates the task based on the message. If the message is not recognized, the “Invalid message” notification will appear on the serial monitor. To test the functionality of the program with implemented threads, I ran it and sent various messages to the monitor to see if the behaviour is as expected.

Testing

To test the implementation of the threads assignment. I will scan the signal on every sprinkler to show that multiple scenes are running at the same time. Graph:



In this graph, It can be seen clearly that two scenes are executing at the same time. Scene 1 starts by turning on sp1 for 10 seconds, after those 10 seconds pass sp2 is turned on for 5 seconds. But as it can be seen scene 3 is also running. First it turns on sp2(before scene 1 turns it on) for 4 seconds, then it turns it off and turns on sp3 again for 4 seconds before turning it off.

Issues

The only issue I faced during implementing the first part of this assignment was that I was getting a watchdog fatal error message after a scene is executed once. At first I thought that maybe I was not allocating enough memory for the task, but after further analysis of my code and the example code in the presentation, I noticed that I was missing the vTaskDelete function call at the end of the tasks I created. When I added it, I no longer encountered the same error again.

Mutex

For the sake of clarity a table of implemented requirements will be shown below:

Requirement	Description
Scenes should not use same sprinklers at the same time	The scenes should be executed in a manner where there is no conflict. For example: scene 1 is using sprinkler 1, but scene 2 also needs sprinkler 1. In this situation scene 2 should wait until scene 1 is done using the sprinkler.

The goal of the mutex is to protect resources that are shared across tasks from the resource being changed by multiple tasks at the same time, as that can cause unexpected behaviour. In this case, the resources that can be shared across different threads are the sprinklers. For this reason I created three separate mutexes for each sprinkler. To handle those mutexes, I introduced the mutexHandler function. This function takes a mutex as an argument and then tries to take the mutex. If the mutex is occupied by another task, the function will get into a while loop printing the message "Waiting for mutex..." on the serial monitor. When the mutex is successfully taken, the function terminates. To properly handle situations where a scene tries to access a sprinkler that is currently using the same sprinkler, I integrated the mutexHandler in the executeScene function. I put the mutexHandler function at the places where a scene needs to turn on a specific sprinkler. Before the sprinkler is turned on, it first must have the mutex to do so, that is why the mutexHandler is called before the sprinkler is turned on. Whenever the scene is done using the sprinkler and turns it off, the mutex is released, giving the opportunity for other scenes to use the same sprinkler.

Testing

To showcase the functionality of the mutexes I will try to run two scenes which need access to one sprinkler at the same time and analyze the behaviour of the program by analyzing the signals of each sprinkler using a logic analyzer. Graph:



I tried to run both scene 1 and scene 2 at the same time. Both of those scenes need to use sp1 at the same time, but the mutex prevents it. As it can be seen in the graph, there is a really short transition at the 10 second mark. This is the timestamp where scene 1 releases sp1 and scene 2 takes it over. This proves that scene 2 waited for the mutex and did not access sp1 until scene 1 was done using it.

Issues

The issue I faced here is that at first I did not know how to check if the mutex has been successfully taken, but after reading the explanation of xSemaphoreTake, I noticed that it returns a Boolean based on

the result of the operation. If the operation is successful, the result is true if not – false. This helped me resolve my issue.

Queues

For the sake of clarity a table of implemented requirements will be shown below:

Requirement	Description
Scenes should not use same sprinklers at the same time	See mutex requirement for description
The function which executes scenes should receive scene steps indirectly	The steps of scenes should not be directly loaded into the function that executes scenes, the steps should be loaded from a “middleman”(in this case queues)
Scenes should not interfere with each other in any way	Scenes should run freely without any interruptions. This means that they should execute one by one in different time frames.

The first step is to create the queue itself. After the queue is created, I need to fill it with the steps of the scene. This is done with the addStepsToQueue function. Whenever a user sends a message corresponding to a scene, the steps of that scene are filled in the queue. This function does that by going through the array and putting the steps in the queue one by one. After, that there is a need for a task that reads the steps from the queue. This task is pretty similar to the executeScene method. The differences in executeSceneFromQueue are that this task first checks if there are any steps in the queue and if there are steps then it executes them one by one. The method of using queues effectively fixes the issue of resource conflict, because queues follow the FIFO(first in first out) principle. This means that steps are executed in strict order.

Testing

To show that scenes are executed in order and scenes do not interfere with one another. I will prove that scenes execute one by one by analyzing the signals of the sprinklers by analyzing their signals. Graph:



In this graph it can be seen that scene 2 starts executing only when scene 1 has finished, despite the fact that the scene 2 message was sent no longer than 3 seconds after the scene 1 message was sent. This is because the steps are loaded into the queue and follow the FIFO principle.

Conclusion

FreeRTOS is a great tool for creating more complex systems that need to maintain multiple processes at the same time. I enjoyed this assignment and I think I managed to get a good basic understanding of how freeRTOS works and its principles.